When Does Admission Control Pay? Decomposing End-to-End Wall Clock in Memory-Bound Agentic LLM Serving

Anonymous Submission

***Abstract*—Agentic LLM sessions are long, multi-round jobs whose KV footprint grows every round, and on a memory-constrained accelerator the pool is the binding resource. Admission control therefore looks like the obvious lever: a controller that reads engine telemetry and admits on measured capacity should beat a static cap. We give a measurement that says how much room there is, and show what happens when you build against it. Throughput during busy time is $\overline{R}/\tau$—running requests over per-step cost—so raising concurrency by a factor f buys $f^{1-\epsilon}$, where $\epsilon = \mathrm{d}\ln\tau/\mathrm{d}\ln\overline{R}$. Near zero, busy-time throughput keeps converting; near one it does not. It is a property of the workload *and* the engine configuration, not of the controller—our controllers run with a host KV tier, and on that configuration it stays below one across their whole operating range, so the ceiling we measure on device-only caps is not what bounded them. It bounds what added concurrency is worth, not what a whole run achieves—idle time and the SLO axis are separate. We measure ϵ on a memory-bound agentic replay (200 sessions, one 24 GB GPU, a pool that holds four median sessions) and get 0.38 to 0.61 depending on which step cost a system can observe: every 10% of added concurrency buys 4–6% of throughput.**

The verdict that criterion returns here is negative, and we report it at that strength. Nine controller variants spanning four routes, plus a reimplementation of an independent congestion-based controller, none is above the static Pareto front at the latency threshold interactive serving cares about—the best ties it on one run and lands 4.6 pp below on its repeat—; a plain cap on resident sessions remains the fastest policy we measured. That is not an argument for admitting everything: with no control at all the median request waits 1,774 seconds for its first token, the prefix cache collapses from a 75% planned hit rate to 1.6%, and SLO falls to 3%. The question is which selective rule, not whether to be selective. Two measurable effects, rather than one we can tune away, account for part of why our variants fail—the arm-by-arm attribution of §V-C covers four of the eleven arms we tested and leaves seven as residuals we can measure and cannot yet explain. They hold nearly half the pool idle while sessions queue—their own headroom estimate on the base configuration is 1.79× the truth (1.0–4.1× across all of them)—and the per-step elasticity measured on the configuration they actually run stays below one across their whole operating range, so the engine had room they did not take: the ceiling we measure on device-only caps does not survive on the host-tier configuration the controllers use. The wall clock splits into busy and idle time; admission policy moves only the running count, and τ rises with it. The idle component (7.4–36.3% of the sampled span) comes from gaps the tool waits set—an agent session issues no request while a tool runs, and that wait averages 16.85 s, not the sub-second figure the trace's median suggests—though how many such gaps a run collects is partly the policy's, since a policy sets how many sessions stay live. On device-only configurations we can account for the rise token for token; on host-tier ones we cannot, and we say so. We also report the six of our own claims we retracted along the way, and the boundary of the verdict: at a loose latency threshold one variant does cross the front, by a margin we cannot resolve against run-to-run noise.

***Index Terms*—LLM serving, admission control, KV cache, agentic workloads, performance characterization, negative results**

I. Introduction

An agentic LLM session is not a request. It is a long-lived job that issues a round of inference, waits on a tool, and comes back—tens of times—with a context that grows every round. On a memory-constrained accelerator this makes admission control look like the obvious lever. The KV pool is the binding resource; a single large session can occupy most of it; and the sessions themselves are wildly heterogeneous. If a scheduler can see how much of the pool a candidate will need and how much room is left, surely it should beat a fixed number.

We believed this, built a controller that does exactly that, and spent a considerable amount of GPU time discovering that on our workload and hardware it is false. This paper reports that negative result, and the negative result is most of what we have. Alongside it we report one measurement—the elasticity of per-step cost with respect to the running count—that we hoped would say in advance how much room admission control has. It does bound that room on the configurations we measured, but when we tested whether it explains our controllers' failure, on their own configuration and on a second workload, it did not. We report it as a diagnostic of a workload, not as a predictor of whether a controller will win.

A. What we measured

We serve a replay of 200 real coding-agent sessions on a single RTX 3090 (Qwen2.5-Coder-7B, 96K context, SGLang, a 101,432-token KV pool) and compare a family of admission policies against a sweep of static concurrency caps. Nine controller variants, spanning four mechanistically distinct ideas, all land on or below the static Pareto front, and so does a reimplementation of an independent congestion-based controller from the literature (Fig. 1). The best of them reaches the front on one run and falls below it when we repeat that run.

B. Why

We decompose the wall clock into *busy* time, during which the engine has at least one request in flight, and *idle* time,

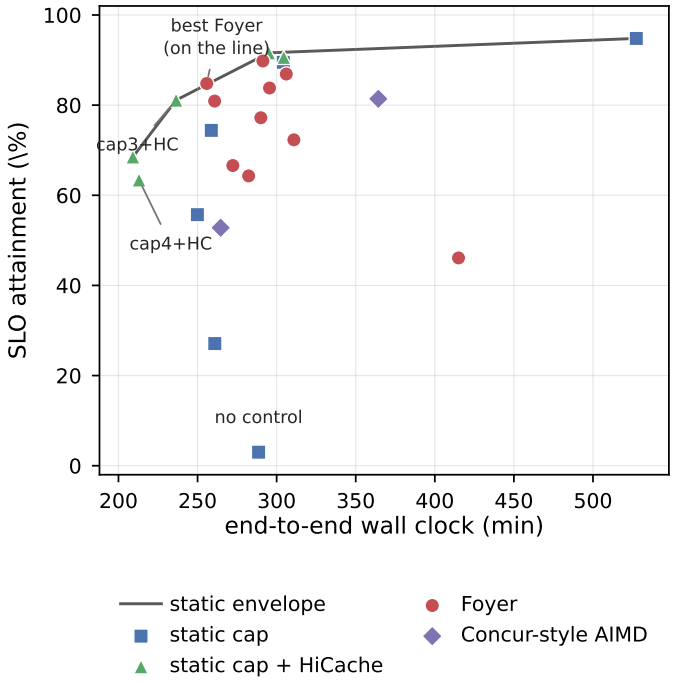


Fig. 1. Every completed run on the (wall clock, SLO) plane. The grey line is the upper-left envelope of the static points; a policy below or on it has not beaten a static cap, because time-slicing two caps approximates any point on the segment between them. Nine Foyer variants plus a reimplementation of an independent controller, none above at the 10 s latency threshold used throughout; §VIII reports how far that verdict is conditional on the threshold.

during which it has none. The busy component is governed by $W_{\text{busy}} \approx (G/\overline{R})\,\tau$ (Eq. 3), where $\overline{R}$ is the mean number of running requests, G the generated tokens and τ the per-step cost. An admission policy moves $\overline{R}$, and on our configurations it moves little else—`bres` is the exception, and §III uses it to show why moving both components at once is still not enough. The difficulty is that τ is not constant in $\overline{R}$: across our runs τ_1 spans 27.0–36.7 ms, and on the device-only family, raising the busy-time running count from 1.00 to 1.84 absorbs a third of what the extra concurrency buys.

The idle component is structural rather than a control failure. A session in a tool wait issues no request, so when all live sessions happen to be waiting, the engine is genuinely empty. The stretches are short in the middle and long in the tail—median 5.6 s, mean 20.6 s—and the tail is what a scheduler feels: 69.6% of all idle *time* sits in stretches over 20 s, an order of magnitude above the 2 s control cycle. Crucially, the tool wait itself is much longer than the trace's median suggests: the median inter-round gap is 0.35 s, but the *mean* is 16.85 s with 22% of gaps over ten seconds. A controller that reads only the median concludes, as we initially did, that sessions never leave and therefore that there is no idle time to recover. There is.

C. Contributions

- We test four routes by which an adaptive admission controller was expected to beat a tuned static cap on this workload, and we are precise about what each one rests on. Two are falsified by measurement against criteria we fixed before the runs—changing the accounting base (§V) and changing the growth reserve. One is a knob rather than a hypothesis, and we report it only because it slides the operating point along the front and adds no mechanism. The fourth, changing the execution granularity from session count to running-request count, is falsified by measurement *and* by an argument about why the permit mechanism cannot express the control it would need. We do not call the last two of these falsifications of a stated hypothesis, because they were not.
- We give a two-part decomposition of the wall clock (§III) that locates the cancellation: admission policy moves the running-request count, and the per-step cost moves with it. On the device-only configurations we can account for that rise token for token—evicted prefixes are recomputed almost one for one—and we report where that account stops: on host-tier configurations the recomputation count is flat while the step cost still moves, and we do not know why (§III).
- We reduce the question to one measurable number, the elasticity of per-step cost with respect to the running count (§IV), and measure it at 0.38 on the model-consistent step cost and 0.61 on the per-request one, on the configurations whose mechanism we can account for token for token. We offer it as a diagnostic rather than a predictor: it says where on the curve a policy operates and how much the remaining headroom is worth, and §IV is explicit about the sense in which it is circular and the sense in which it is not.
- We characterize when adaptivity *could* have paid, and show that our workload is not that case: the same throttle configuration that is worth $+51\%$ at 25 sessions costs -42% at 200 (§VI). Put differently, no single scalar cap is optimal across both, which is the regime adaptive control exists for. On the low-load tier, where that argument bites hardest, the controller is *not* worse—same-generation measurements put it on the front, $+0.9$ pp above the line and inside the noise. On the high-load tier it is dominated. That asymmetry, not a blanket failure, is what this paper measures.
- We make the measurements reproducible: the front and each controller's signed distance are produced by one script from the raw per-step logs, the tables and the sub-thousand-step runs by two others, all named in §VIII-0e. We report the six claims we had to retract along the way, with the checks that caught them.

D. What we do not claim

We do not claim that admission control is useless in general. We claim it does not pay *here*, we show the measurement that decides it, and we argue (§IV) that the same measurement can be taken on another workload before committing to a sweep. We also do not claim our hardware and model are

representative; our results are from one 24 GB accelerator and one 7B model, and extending them is future work.

II. Background and Setup

A. Workload

We replay 200 coding-agent sessions drawn from the TraceLab corpus [1]. Each session is a sequence of rounds; a round presents a prompt of median 23,187 tokens (p90 51,064; maximum 89,436, which alone is 88% of the KV pool), and between rounds the session waits on a tool before returning with a longer context. Arrivals are Poisson at $\lambda = 0.04\,\mathrm{s}^{-1}$ over an 80-minute window, giving 999 steps in total. Two sessions of maximum size cannot be co-resident: their combined contexts exceed the pool by 76%.

The tool wait is the load's defining property and the one most easily misread. Its *median* is 0.35 s, which invites the conclusion that sessions never leave the engine. Its *mean* is 16.85 s and the maximum is 300 s; 21.8% of the tool waits the trace specifies exceed ten seconds, which is 14.4% of all round transitions once the zero-wait ones are counted, or 18.6% on the measured gaps. We verified that our replay harness honors the trace faithfully: across 799 round transitions the measured gap is 16.85 s mean against a demanded 15.82 s. On a typical transition the harness adds nothing—the median of the per-transition measured/demanded ratios is 1.001—while the ratio of the two means is 1.065, the difference sitting in the tail. The corpus's own characterization makes the same point qualitatively—it reports "diverse and heavily-tailed tool calls" and calls these gaps "human-paced"—but gives no numeric mean [1], so the 16.85 s above is our measurement over the released trace, not a figure we quote from that paper.

B. Engine and the binding constraint

We use SGLang v0.5.10 [2] with Qwen2.5-Coder-7B-Instruct [3] extended to 96K context, on one RTX 3090 (24 GB). KV is paged [4]; at `mem-fraction-static` 0.85 the pool is 101,432 tokens. Hierarchical caching (HiCache), with a host tier of twice the device pool, is enabled on the configurations marked *+HC* and on every Foyer run; the five plain static caps run without it. We keep both because the host tier turns out to change the mechanism (§III). Every run is guarded: if the pool does not come out at $101{,}432 \pm 20$—which happens when another tenant holds memory on the card—the run is aborted rather than recorded, because SGLang sizes the pool from free memory at startup and a shrunken pool silently produces incomparable numbers.

The relevant consequence is that the pool holds only about four median sessions. Concurrency is therefore capacity-bound, not scheduler-bound, and this is the premise on which everything below rests.

C. The controller under test

Foyer is an admission controller that reads engine telemetry and issues *named* permits rather than a scalar cap: a session is admitted by appearing in a permit file, and the runner refuses to start a round for a session that is not listed. This design choice matters for the comparison, because it is what allows per-session actions at all—including the pause primitive we exploit in §V-H. It also means that a session, once admitted, holds its permit for its entire lifetime, so the controller never frees capacity by discarding a session's state—it admits fewer sessions instead. That is a statement about the *policy*, not about the memory: the engine's radix cache still evicts parked prefixes under pressure, and we measure that it does and that they are recomputed (§III). We return to this distinction in §VIII, because it bounds how far our negative result generalises.

Each control cycle (2 s) the controller computes

$$\Pi \;=\; \max(\rho, \textstyle\sum_i c_i) \;+\; \sum_i g_i, \tag{1}$$

where ρ is the engine's reported occupancy, c_i the context length of live session i, and g_i its forecast growth over the next `horizon` rounds. A session is admitted when $\Pi + n \le B$, with n the candidate's estimated need and B the budget. The three terms are respectively the measured physical occupancy, a reservation for sessions currently parked in a tool wait (whose KV is evictable and therefore absent from ρ), and a reservation for future growth.

D. Baselines and the verdict criterion

Baselines are static caps: at most N sessions resident, $N \in \{1, \ldots, 5\}$, with and without HiCache. We compare on two axes: end-to-end wall clock (time from the first submission to the last completion) and SLO attainment (fraction of requests whose time-to-first-token is under 10 s).

The verdict criterion deserves care, because "faster than cap-N" is not the question. The question is whether a policy reaches a point *above* the static front. A point that lies merely between two static caps is not a win: mixing cap-N and cap-M by time slice approximates any point on the line between them. We flag this segment argument as an argument, not a measurement—we did not run the time-sliced policies, and carrying queue state and cache contents across a switch could bend the line. It is the criterion we apply, and it is the stricter of the two we could have used—stricter than comparing each controller against the discrete static points alone, because the line sits above them. We state the direction plainly rather than claim it as a caution: this choice makes our negative result *easier* to reach, not harder. Our best arm is $+0.22$ pp against the line and would be $+3.7$ pp against the nearest static point below it in wall clock. So we take the upper-left envelope of the static points and report each controller's signed distance above or below it. We classify a signed distance of more than $+1.0$ pp as above the front, less than -1.0 pp as below it, and the band between the two as *on* it—a tie, not a win. The band is a convention we fixed before running anything, and we report it here because every verdict in this paper depends on it. Our later replicate measurements (§V) bracket it rather than justify it: the SLO spread of configurations we ran more than once ranges from 0.5 pp to 5.1 pp, so for some configurations the band is generous and for others it is far too narrow. We flag this as a weakness of the criterion, not a defence of it. One further

property of the envelope deserves flagging, because four of the nine controller variants finish slower than 295 minutes. Its right-hand segment runs from `cap2`+HiCache at 295 minutes to `cap1` at 527, and `cap1` is the only anchor there that runs *without* HiCache. Those four—`hz2`, `bres`, `breshz1` and `foyfix`—are therefore judged against an interpolation between one host-tier point and one device-only point. If the envelope is drawn from the host-tier anchors alone, that segment does not exist and those four cannot be adjudicated at all; drawn from device-only anchors alone, they sit further below. We report the conservative reading and note that the honest statement about those four is *below the front as we defined it*, not *below every static configuration*. The front and every controller's signed distance are computed by one script from the raw per-step logs; the table generator computes the distance for the four runs that did not complete all 999 steps—both are named in §VIII-0e—and that script's own first version was wrong: it extrapolated past the end of the envelope and declared the slowest static cap a winner. We fixed it and report the episode because it is the failure mode this criterion is most prone to.

III. Decomposing the Wall Clock

A. Two components

We split the wall clock at every sampled instant according to whether the engine has any request in flight:

$$W = W_{\text{busy}} + W_{\text{idle}}. \tag{2}$$

Fig. 2 shows the split for every completed run. Two things are immediately visible and both matter later.

First, idle time is not noise. It ranges from 7.4% of the fastest configuration's sampled span to 36.3% of the slowest's—the denominator here is the span the sampler observed, which runs about six minutes longer than the request wall clock of Table I, so the fractions against that wall clock are 7.6% and 36.7%—and it is not spread evenly. Over 1,849 stretches the median is 5.6 s—close to the 5 s sampling floor, because most tool calls are quick—but 17.6% of them exceed 20 s, the 90th percentile is 30.1 s, and the longest is 511 s (Fig. 3). The tail, not the middle, is what a scheduler feels, and it sits at the scale of the 16.85 s mean tool wait.

Second, the two components move independently. `bres` (§V) reduces idle from 15.9% to 10.1%, recovering about 17 minutes—and is nonetheless *slower* than its own baseline, because its busy time grows by 21 minutes. Any account that treats "more concurrency" as one thing will get this wrong.

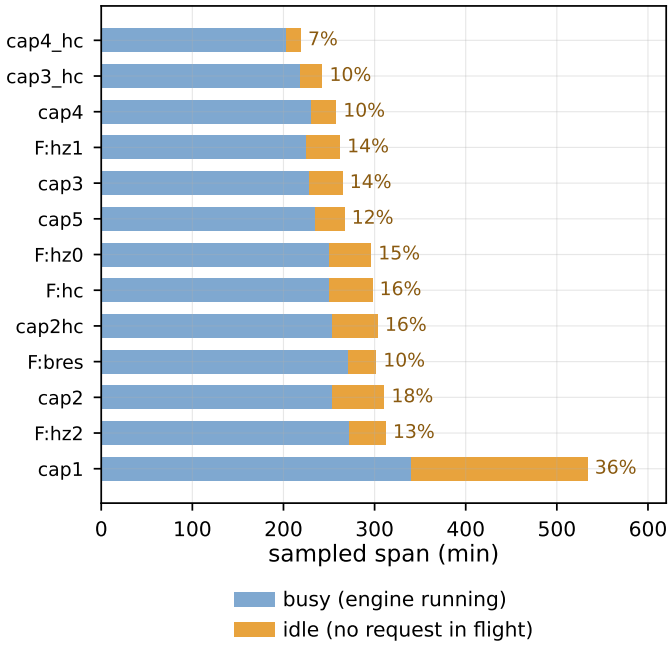


Fig. 2. Wall clock split into busy and idle for every completed run, ordered by total duration. Idle is 7.4% for the fastest configuration and 36.3% for the slowest. Note `bres`: it cuts idle to 10.1% and is still slower than its own baseline, because busy time grows instead.

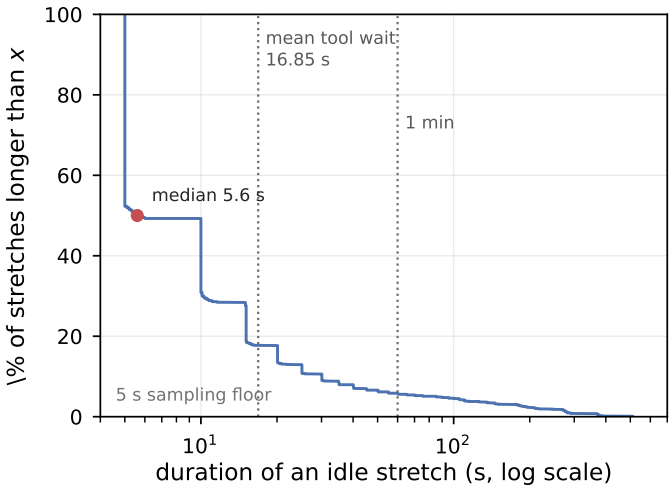


Fig. 3. Survival function of idle-stretch duration, pooled over all completed runs (1,849 stretches). Most stretches are short—the median is 5.6 s—but the minutes are not: 69.6% of all idle *time* sits in stretches longer than 20 s, and the mean stretch is 20.6 s, the same order as the 16.85 s mean tool wait. The cliff is the 5 s sampling floor.

B. What determines busy time

During busy time the engine executes steps; each step produces one token per running request. Hence

$$W_{\text{busy}} \approx \frac{G}{\overline{R}}\tau, \tag{3}$$

with G the total generated tokens (fixed by the trace at 706,010) and τ the per-step cost. Across our runs τ is not a fit parameter we tuned; we obtain it two independent ways, and reporting both is what shows they are measuring different things rather than confirming each other.

a) Method 1 (derived).: $\tau_1 = W_{\text{busy}} \cdot \overline{R}/G$. Two choices in that expression are easy to get wrong and both matter. The time term must be the *busy* component, not the wall clock—using the wall clock charges idle time to the steps. And $\overline{R}$ must be the mean running count *during busy time*, not the mean over the whole run: the whole-run mean includes the idle samples,

where the count is zero, so it equals $\overline{R}_{\text{busy}}\times$ (busy fraction). Using it makes τ_1 too low by exactly the busy fraction—36.3% for `cap1`, whose idle share is 36.3%. We report both columns in the data, and use the busy-conditional mean throughout; the whole-run mean is what an earlier revision of this paper used, and it made the two methods agree far better than they do.

b) Method 2 (independent).: For each step record we compute the per-request decode interval, $(\text{total_duration} - \text{first_token})/(\text{output_len} - 1)$, and take the median over steps with $\text{output_len} \geq 32$.

The two do not agree exactly, and the disagreement is informative rather than embarrassing. τ_1 exceeds τ_2 on every run, by 7.9–39.8%, which in absolute terms is 2.6–8.2 ms of a 27–37 ms step. The gap is *consistent with* engine steps that are prefill and therefore emit no output token—we can bound it, not identify it, because subtracting a median from a mean does not isolate a component. All we can form from these two is their ratio: the excess $\tau_1/\tau_2 - 1$ runs 17.6–39.8% across the five device-only configurations. Two things follow, and we separate them. First, it is a ratio of *times*, not of steps, and we have no measurement of the fraction of step *counts* that are prefill—chunked prefill makes those two different numbers. Second, even as a time ratio it is the gap between two measurements, not a decomposition of one: it is consistent with prefill and non-decode work filling it, but we cannot rule out that some of the difference is the two estimators disagreeing for other reasons. So the two are not two estimates of one number; they are measurements of two different things, and we do not size a component with their difference. Fig. 4 shows both, run by run. Method 2 remains the one that does not depend on the quantity it is used to explain, and it is the one we quote when the question is the cost of a decode step. The engine's own latency counters (`itl_p50`, `e2e_p50`, `ttft_p50`) are exported as zero by SGLang v0.5.10 in our configuration, so no directly measured step time exists in our data; instrumenting this is future work.

C. The per-step cost is not constant in concurrency

Equation 3 would be uninteresting if τ were a constant. It is not. Across our 13 runs τ_1 ranges from 27.0 to 36.7 ms and τ_2 from 20.6 to 33.1 ms, and both rise with the running count: fitting Eq. 4 to the five device-only configurations gives $\varepsilon = 0.38$ for τ_1 and 0.61 for τ_2 ($R^2 = 0.89$ for the latter, Fig. 5).

The relation is a trend, not a law. Two host-tier configurations with similar running counts behave differently: `cap4_hc` runs 1.73 requests at 29.8 ms while `F:hz0` runs 1.66 at 35.7 ms. Running count sets a floor and a trend, not a value—and on the host-tier family, as we show below, it does not set the value at all.

This is the crux of the paper, and we state it at the strength the arithmetic supports. Hold τ_1 at its value for `cap1`, 28.8 ms, and busy time falls as $1/\overline{R}$: raising the running count from 1.00 to 1.84 should have cut busy time by a factor of 1.84, from 340 minutes to 184. The measured curve flattens and lands at 234. The rise in τ takes back 50 of the 156 minutes

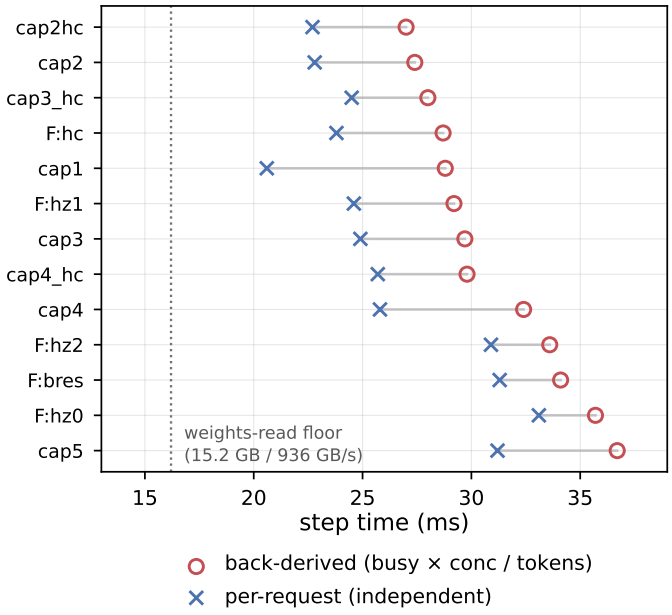


Fig. 4. Per-step cost by two independent measurements, one run per row: circles are model-consistent (all engine steps), crosses are the per-request decode interval. They do not agree—τ_1 exceeds τ_2 by 8–40%— and the gap is consistent with the prefill share, not measurement error; §III bounds that share rather than identifying it. The dotted line is the cost of reading the weights once; every run sits above it.

that the extra concurrency would otherwise have bought—*a third* of the gain, not all of it and, as an earlier revision of this paper claimed, not more than half. The other two thirds are real: raising concurrency does make the engine faster. What it does not do is make it faster in proportion, and the shortfall compounds with every controller that tries to exploit the difference.

a) What the engine is doing when it is not decoding.: SGLang exposes two counters that answer this directly: tokens evicted from the radix cache, and prompt tokens that had to be prefilled rather than reused. Across the five static caps *without* HiCache the two track each other almost exactly. `cap1` evicts 9.2 M tokens and prefills 8.6 M uncached; `cap5` evicts 21.3 M and prefills 20.7 M; the gap is 0.6–0.7 M on every one of the five. An evicted prefix is recomputed, close to one token for one, and that recomputation is chunked into decode steps. Those same five runs give recomputation rising with τ across four of the five points: 9.2 M uncached tokens at 27.4 ms, 11.5 at 29.7, 15.4 at 32.4, 20.7 at 36.7. The exception is the first point, `cap1`, which recomputes least (8.6 M) and still costs 28.8 ms—more than `cap2` does for more recomputation. We report the ordering we have rather than rounding it to a monotone one: it is a mechanism consistent with the τ trend on four of the five device-only configurations, and one point it does not order. It is five points on five different policies, not an intervention: we did not hold concurrency fixed and vary eviction, so we call it consistent rather than demonstrated.

b) Where the mechanism stops.: With HiCache enabled the correspondence breaks—and it breaks in a way that con-

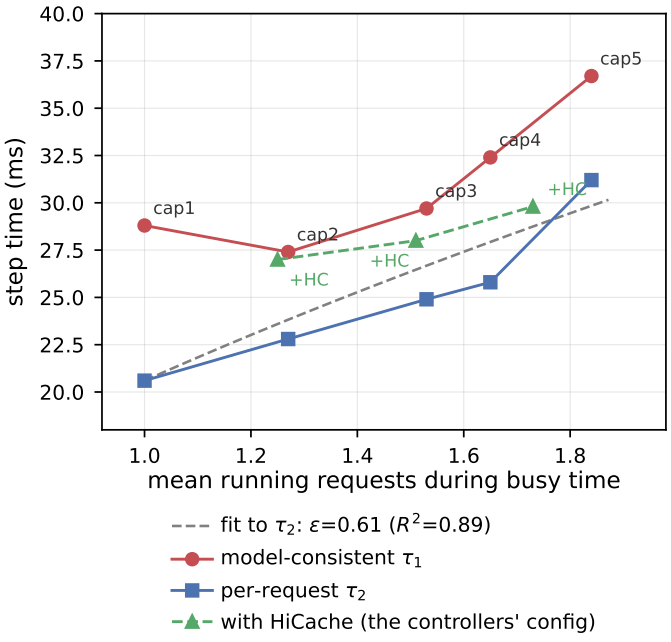


Fig. 5. Per-step cost against the running count during busy time. Solid circles are τ_1 and squares τ_2 for the five device-only configurations; τ_1 is model-consistent (it averages over all engine steps, including prefill that emits no output token), τ_2 is the per-request decode interval, and τ_1 exceeds τ_2 by 8–40%. The green triangles are τ_1 for the three HiCache configurations—the ones the controllers run—and they are drawn against the same axes to make §IV's point visible: over $\overline{R}_{\text{busy}}$ 1.25 to 1.73, which covers the whole controller operating range, the host-tier curve rises 27.0 to 29.8 ms while the device-only curve over a comparable span rises 27.4 to 36.7. We plot τ_1 only for the HiCache points to keep the figure readable. The fitted exponent is the elasticity ε of Eq. 4; it is fit to the device-only points alone.

firms the mechanism rather than undermining it. The device cache misses as much as before: `cap4_hc` misses 22.4 M tokens on the device, more than the device-only `cap4` misses. But only 9.1 M of that is recomputed; the other 13.3 M is served from the host tier. Across the eight host-tier runs the host serves 13–19 M tokens each, and across the five device-only runs it serves *exactly zero*—the device miss equals the recomputed count to within rounding on all five. Recomputation is still what the device cannot absorb; the host tier is a second absorber in front of it.

That does not resolve the Foyer family, and we say so plainly. Those runs sit at a flat 8.6–8.7 M recomputed tokens while τ_1 ranges from 27.0 to 35.7 ms and τ_2 from 22.7 to 33.1 ms. Whatever moves τ there is not recomputation, and we do not know what it is. We flag it as the largest open question in the paper, and note that the negative result does not rest on it: the cancellation is measured directly, not inferred from the mechanism.

c) A refutation we had to withdraw.: An earlier revision withdrew this mechanism outright, on two checks that were both wrong. The first was that total prefilled tokens equalled the trace's planned prefill to a ratio of 1.00, so no token could have been computed twice. `prompt_tokens_total` counts the full submitted prompt *including* its cached portion, so it equals the sum of prompt lengths by construction and cannot detect recomputation; the quantity that can is the uncached count used above, which varies by 2.4× across these runs. The second was that SGLang reported no retractions. Radix-cache eviction is not a retraction—retractions count running requests preempted mid-flight, a different event with its own counter. We record the episode because it is the same failure mode as the segment argument of §II: a check that looks like a measurement but is an identity.

D. Why idle time exists, and why recovering it does not pay

The length of an individual idle stretch is a property of the workload, not of the controller. A session in a tool wait holds its permit and its KV but issues no request; with only about two live sessions, the probability that all of them are simultaneously waiting is substantial. The direct evidence is the duration of the stretches themselves: they are of the same order as the tool waits, and an order of magnitude longer than the 2 s control cycle (Fig. 3). No controller can shorten a gap that the tool set.

How much idle a *run* accumulates is a different matter, and there the controller is not off the hook. Idle accumulates when every live session happens to be parked at once, and how many sessions are live is precisely what an admission policy sets. Two configurations with the same mean running count make this visible: `horizon`=1 (1.52) and `cap3`+HiCache (1.51) idle 36.7 and 24.1 minutes respectively—a 52% difference at matched concurrency—and 12.6 of the 19.4-minute wall-clock gap between them is that idle difference rather than step cost. So the gap durations are the workload's, and the number of gaps is partly the policy's.

We note one claim we had to drop here. An earlier draft reported that idle fraction falls monotonically with mean concurrency. With the full data it does not: `F:hz0` at 1.41 running requests idles 15.2% while `cap4` at 1.48 idles 10.4%. The weaker statement that survives is the one we use—the stretches are structural and long.

The obvious remedy—keep more sessions live so that someone is always ready—is what `bres` does. It works in the narrow sense: `bres` recovers about 17 of the minutes that its own baseline spends idle. It is still slower overall, because the sessions it keeps live raise the step cost by more than the recovered idle is worth. Idle is not unrecoverable; it is recoverable at a price that is not worth paying here. The structural remedy would be to make the engine do work during a wait, but there is no work to do: the session's next round does not exist until its tool returns.

IV. A Criterion for When Admission Control Can Pay

The decomposition turns the question into one number. Throughput during busy time is $\overline{R}/\tau$, so raising concurrency by a factor f buys a factor $f^{1-\epsilon}$, where

$$\epsilon = \frac{\mathrm{d}\ln\tau}{\mathrm{d}\ln\overline{R}} \tag{4}$$

is the elasticity of per-step cost with respect to the running count. At $\epsilon = 0$ the step cost is flat, concurrency converts one for one into throughput, and an admission controller has room to work. At $\epsilon = 1$ the two cancel exactly and it has none. Everything in between is a quantitative statement about how much room.

On the five device-only configurations, where §III establishes that the step cost is driven by recomputation, $\overline{R}_{\text{busy}}$ runs from 1.00 to 1.84. A least-squares fit of Eq. 4 over those five points gives $\epsilon = 0.61$ on the per-request measurement ($R^2 = 0.89$), or 0.38 on the model-consistent one (Fig. 5).

That fit is a straight line through a curve, and the curve is what matters. The segment elasticities are $\epsilon = -0.21$ from `cap1` to `cap2`, $+0.43$ from `cap2` to `cap3`, $+1.15$ from `cap3` to `cap4`, and $+1.14$ from `cap4` to `cap5`. Below $\overline{R}_{\text{busy}} \approx 1.5$ added concurrency still buys something; above it, ϵ passes one, which means the engine gets *slower* in wall clock as more requests run. The single number 0.38 averages those regimes and describes none of them.

The operating region is the one that matters, and it is not the region the controllers run in. All nine controller arms run with HiCache enabled, which the five device-only caps above do not. Measuring the same elasticity on the three HiCache configurations we have gives $\epsilon = +0.19$ from `cap2`+HC to `cap3`+HC and $+0.46$ from `cap3`+HC to `cap4`+HC, over $\overline{R}_{\text{busy}}$ 1.25 to 1.73. That range covers the whole controller operating region (1.35 to 1.69), and over it ϵ never approaches one. At matched running counts the HiCache caps pay 1.7 ms less per step at $\overline{R}_{\text{busy}} \approx 1.5$.

This contradicts what an earlier draft of this paper claimed, and it changes what the criterion is for. The cliff in the device-only curve is a recomputation effect: §III measures that those runs recompute what they evict, while under HiCache the host tier absorbs it. The cliff does not survive on the configuration our controllers actually use. So the criterion does *not* explain why they lose—on their own configuration they were not in a region where added concurrency was self-defeating, and the engine had room they did not take. What the measurement does is rule one explanation out. The failure is not that this workload denies admission control its lever; it is that our controllers do not pull the lever they have, which is what §V-B and the attribution of §V-C measure directly.

We are explicit about what this does and does not establish. The HiCache curve rests on three configurations and two segments, one run each, with no error bars, and it stops at 1.73—we cannot see whether a cliff appears beyond the controllers' region. A $\pm 10\%$ perturbation of τ puts the segment elasticities' bounds across one even on the device-only curve, so the cliff is a description of the fitted points rather than a threshold we have located precisely. And it is measured on different policies rather than by an intervention, so it is a cross-policy association. On our data it is also computed from the same wall clock the model then explains—τ_1 is derived from W_{busy}, so using it to predict W_{busy} restates rather than tests—and its points are different policies, so the slope is a cross-policy association, not a transportable causal derivative. τ_2 is genuinely independent of the wall clock but measures only decode, so it cannot be dropped into a throughput equation that includes prefill.

We tested whether the curve transfers to a second workload and it does not obviously do so. On a real agent trace (117 sessions, tool waits of seconds rather than the 95 ms of the synthetic trace, three static caps and one controller arm), the model-consistent elasticity is negative across the measured range—-2.40 then -0.28, at $\overline{R}_{\text{busy}}$ 1.08 to 1.29—and the controller arm there runs at 1.22, below every one of the nine arms above, and still does not beat the static front. The per-request measure has the opposite sign on that trace ($+0.41$, $+0.48$). We report this as a boundary: the criterion is a description of one workload on one configuration, and where we could check it elsewhere it neither transferred nor explained the failure. It is not, as an earlier draft claimed, something that decides the question in minutes before building anything.

V. Results: Four Routes, One Outcome

A. The frontier

Fig. 1 places every completed run on the (wall clock, SLO) plane and draws the upper-left envelope of the static points. Table I gives the numbers and each controller's signed distance from that envelope.

The result is unambiguous. Of nine controller variants, none is above the envelope. The best, a growth-reserve setting of one round instead of the default three, lands $+0.22$ percentage points above the line. We ran it again, and the repeat lands at -4.57. Same code, same trace, same pool, guarded the same way: 260.8 minutes against 255.8, and 80.9% SLO against 84.8%. *The one arm that reached the front does not reproduce reaching it*, and the two runs of that single configuration differ in signed distance by 4.8 pp—one a tie, the other clearly below.

That is not an isolated unlucky pair. Four configurations have been run more than once, and while their wall clocks are stable across repeats—coefficients of variation of 0.9%, 1.0%, 1.6% and 1.8%—their SLO is not: the ranges are 3.9, 5.1, 1.0 and 0.5 pp. The two widest belong to configurations that *decide* something—the leftmost envelope anchor and the best controller arm—so the front we compare against, and the best result we report against it, are both uncertain at a scale larger than the ± 1.0 pp band we judge with. A fifth pair, two runs of `cap3`+HiCache, we do not use: one of them exceeded its own cap. Against that, the best controller's $+0.22$ pp is not merely inside the noise—it is smaller than the noise of the anchors by an order of magnitude. We record it as *on* the line rather than above it, and we state plainly that resolving a margin this small would need several repeats per configuration, which we did not run.

The same repeat moved the frontier and changed nothing. The leftmost anchor's second run came out both faster (209.1 against 213.0 minutes) and higher on SLO (68.5 against 63.4), so it displaces the first as the envelope's leftmost point. Not one controller verdict moves: every one of them finishes to the right of the next anchor at 236 minutes, so none of them is judged against the segment that changed. We report both runs

rather than keeping the better one, because a frontier whose anchors are single measurements is the weakness this whole subsection is about.

The other end of the front deserves its own sentence, because the rest of this paper is negative and a reader could close it thinking the answer is to do nothing. The run with *no* admission control—every session admitted the moment it arrives—finishes in 288.6 minutes at 3.0% SLO, 87.5 pp below the envelope and the worst SLO we report. Its median time-to-first-token is 1,774 seconds, its prefix cache hit rate collapses from the planned 75.5% the runner prints in its end-of-run summary (the all-steps convention, a different one from the round-1+ numbers in §VII—for the same trace `gen_table.py` reports 73.8% on that convention) to 1.6%, and its segment SLO is 30% in the first tenth of the run and exactly zero in each of the nine after that. Uncontrolled admission does not degrade gracefully; it destroys the cache and then serves almost nobody. The question this paper addresses is therefore not whether to admit selectively, but which selective rule to use.

Two features of the front are worth noting before the negative results. The envelope is steep between 213 and 236 minutes (0.76 pp per minute) and shallow thereafter (0.18 pp per minute), so the cost of buying SLO by retreating from the fast end is high. And W is not monotone in concurrency: five sessions (260.9 min) is *slower* than four (249.9 min)—the same configuration family, no host cache on either side—because the per-step cost rises faster than the request count does. We return to this non-monotonicity in §III, where it turns out to be the first appearance of the mechanism that drives the whole frontier.

B. The controllers leave memory unused

Before the routes, one measurement that reframes all of them. Our controller keeps every session's permit for its lifetime, so its pool occupancy is a policy output, not a constraint. It runs at a median of 49.2% of the pool on the base configuration, and the median number of sessions queueing for admission is 66. The configuration that finishes fastest—the repeat of that same static cap of four, at 209.1 minutes—runs at 71.9%, and its first run at 70.2%. Occupancy and running requests track each other reliably across the arms we have (the two columns in `occupancy.csv` rank the twenty arms almost identically), but running count does *not* order wall clock: `horizon`=2 runs 1.25 requests against the base arm's 1.14 and finishes 15 minutes slower. That gap is what the next subsection is about.

The controller believes it is nearly full. Its own decision log records the projection it admits against—the largest of measured occupancy and the summed contexts of live sessions, plus a reserve for their growth—at a median of 89,073 tokens against a budget of 96,360, i.e. 7% free. Its measured occupancy at the same instants is 49,875 tokens: 48.2% of that budget, or 50.8% of the pool, is idle. The projection is 1.79× the truth on this configuration. Across the seventeen controller runs we have projection logs for, the ratio runs from 1.24 to 4.11 with a median of 1.59 (`projection.csv`), so 1.79 is this arm's value, not a constant of the design; the set is named here because the median moves with it—over every run in that file it is 1.40. Two terms produce the gap: charging every live session its full context, when a parked session's KV is evictable and therefore absent from the engine's count, and the growth reserve, which alone is 26,762 tokens, 28% of the budget.

TABLE I

ALL COMPLETED RUNS. *Conc.* IS THE TIME-WEIGHTED MEAN NUMBER OF *live* SESSIONS, WHICH IS NOT THE RUNNING-REQUEST COUNT $\overline{R}$ USED IN §III—A LIVE SESSION IS PARKED IN A TOOL WAIT MOST OF THE TIME, SO `CAP5` SHOWS 4.89 LIVE AGAINST 1.84 RUNNING. Δ IS THE SIGNED DISTANCE FROM THE STATIC ENVELOPE AT THAT WALL CLOCK; ABOVE $+1.0$ PP COUNTS AS A WIN, BELOW -1.0 PP AS A LOSS, AND THE BAND BETWEEN AS A TIE (§II). FOUR RUNS STOPPED SHORT OF 999 (`HZ0` 995, `HZ2` 997, `RG4` 998, AND THE NO-CONTROL BASELINE 997); THEY ARE LISTED WITH THEIR MEASURED VALUES. WE FLAG THIS BECAUSE OUR VERDICT SCRIPT EXCLUDES RUNS BELOW 999 STEPS, SO THE Δ COLUMN FOR THESE FOUR COMES FROM THE TABLE GENERATOR RATHER THAN FROM THAT SCRIPT.

Policy	Wall (min)	Conc.	SLO (%)	Δ (pp)
Static caps				
no control	288.6	1.69	3.0	−87.45
cap1	527.4	1.00	94.8	*anchor*
cap2	304.2	1.99	89.5	−2.23
cap3 (fixed)	258.7	2.89	74.4	−10.70
cap4	249.9	3.94	55.7	−27.82
cap5	260.9	4.89	27.1	−58.39
Static caps + HiCache				
cap2+HC	295.0	1.99	91.6	*anchor*
cap3+HC	236.4	2.93	81.1	*anchor*
cap4+HC	213.0	3.93	63.4	−6.90
cap4+HC (repeat)	209.1	3.87	68.5	*anchor*
Foyer controller variants				
horizon=1	255.8	2.58	84.8	+0.22
horizon=3 (base)	291.3	2.06	89.8	−1.14
horizon=1 (repeat)	260.8	2.69	80.9	−4.57
horizon=2	306.1	2.22	86.9	−4.85
resident base	295.5	2.40	83.8	−7.81
horizon=0	290.0	2.69	77.2	−13.50
horizon=1, no hw	310.9	2.77	72.3	−19.52
no gate, no resv.	272.3	3.25	66.6	−20.93
running gate=4	282.3	3.03	64.3	−25.02
all throttles off	415.0	2.58	46.1	−47.15
Independent adaptive controller (§V-G)				
AIMD, paper params	364.3	1.91	81.4	−11.15
AIMD, retuned	264.6	3.87	52.8	−33.35

Two things follow, and we are careful about the second.

The first is that the controllers under-use the pool, and that this is a property of their estimates rather than of the hardware: the base configuration holds nearly half the pool idle (48.2% of its budget, 50.8% of the pool) with 66 sessions queued, because it believes it has 7% free when it has 48%. Any account of why it is slow that does not mention this is incomplete.

We then asked what actually stops it, by instrumenting the admission decision. The path is a conjunction: a session is

authorized only if the SLO valve is open, the round gate is not full, and its projected footprint fits what remains of the budget. Over the base run's 8,196 blocked cycles—cycles with a session waiting and none admitted—the valve accounts for 6.8%, the round gate for none, and a projection already above budget for 38.4%.

The remaining 54.8% are cycles where the controller selected no candidate at all, and here we are explicit about the limit of what we can resolve. The controller is single-flight by default: it authorizes at most one session per two-second cycle and waits for that session to begin a round before authorizing the next. Whether that gate held or whether a candidate was tried and did not fit is *not distinguishable from our logs*—the controller does not record it—so we do not attribute the bucket to either. One arm bounds it instead: `sf=0` disables single-flight, so the gate cannot be the cause there, and its unexplained remainder falls to 29.6% against a median of 51.4% across the eighteen single-flight arms. That puts the single-flight path at roughly twenty points and leaves the headroom estimate as the largest cause we can measure.

Across the eight Foyer variants that left controller logs—the set is named in `gate_attrib.csv`, and it is eight rather than nine because `foyfix` and the two AIMD arms run a different controller and log no gate state—the projection is over budget in a median 32.2% of blocked cycles (19.5% to 50.8%) and the valve in a median 32.7% (6.8% to 67.9%). Both medians move with the set, which is why we name it: over the wider group of runs in that file they are 37.6% and 10.5%. The round gate never binds except in the two runs that enable it—`rg2` at 29.3% and `rg3` at 18.0% of blocked cycles, against 0.6% or less everywhere else—which is what one expects of a gate that is off by default. We also record what this accounting omits, because it changes the picture: the controller has a starvation guard that bypasses the budget entirely once the oldest waiter has waited 240 s, cooling down for 300 s. On the base run it accounts for 24.8% of all admissions (51 of 206), and while the projection is over budget it is the *only* way in—of 3,700 such cycles, nine admitted, and all nine were forced. It is therefore the guard, not the budget test, that sets the floor on how fast this controller can admit.

The second is that under-use does *not* order the outcomes, so it is not the whole explanation either. `horizon=1` and `cap3`+HiCache sit at the same median occupancy (57.8% of the pool) and the same mean running count (1.52 against 1.51 on the busy-time measure of §III), and the cap is 19 minutes faster. `hz0` runs at 64.1%, higher than that cap, and is 54 minutes slower. Occupancy buys running requests reliably—the two columns in `occupancy.csv` track each other across all twenty arms—but converting running requests into wall clock is not simply the per-step cost of §III. Decomposing that 19 minutes into its idle and busy components, 12.6 of them are idle time—65%—and only 6.8 are busy; the two runs' step costs differ by 1.2 ms (29.2 against 28.0), about 4%. A parked session is one the controller has authorized but not yet released, so the same policy that sets the running count also sets a large part of the idle time. Both effects are real; on this workload the unused-memory one is larger for the arms that finish worst, and the step-cost one is what separates the arms that finish close together.

C. What the two mechanisms explain, arm by arm

We have offered two mechanisms, and the honest thing is to say what each one covers. For every controller arm we took the static cap closest to it in wall clock and compared their occupancy and their step cost. An arm is attributed to the unused-memory mechanism if it runs meaningfully fewer requests than its comparison, and to the step-cost mechanism if it runs a comparable number at a measurably higher cost. Everything else is a residual.

Arm	Occ.	$\overline{R}$	τ_1	Attribution
`horizon=1`	57.8	1.524	29.2	residual
Foyer base	49.2	1.353	28.7	residual
`horizon=2`	54.7	1.439	33.6	step cost
`horizon=0`	64.1	1.666	35.7	step cost
`bres`	56.9	1.481	34.1	step cost
`breshz1`	61.4	1.632	—	residual
no gate, no resv.	62.1	1.678	—	unused mem.
running gate=4	62.3	1.689	—	residual
all throttles off	39.2	1.675	—	residual
AIMD, retuned	66.6	1.706	—	residual
AIMD, paper params	40.3	1.357	—	residual

Four of the eleven are attributed. Seven are not: their occupancy and their step cost both sit close to their comparison cap's, and their shortfall is real but we cannot say from these measurements which mechanism produced it. We publish the table rather than a summary because a summary would claim more than we have—and because "two mechanisms explain the negative result" would be the wrong summary. Two mechanisms explain part of it, on four of the eleven arms; the rest is a measurement we can make and cannot yet interpret.

D. Route 1: the accounting base

Our controller's projection (Eq. 1) charges a session for its full context even while it is parked in a tool wait, when its KV is evictable and therefore absent from the engine's reported occupancy. We argued this was a double charge and removed it, admitting on engine telemetry alone.

The charge is not double. Removing it did admit more—median engine occupancy rose from 49.2% to 56.9% of the pool—and the round-1+ prefix hit rate held at 80.2% against the 80.6% of the run that keeps the charge, so no eviction spiral occurred. But the run was 4.2 minutes *slower* and lost 6.0 pp of SLO. Per §III, the extra occupancy raised both $\overline{R}$ (1.35 to 1.48) and τ_2 (23.8 ms to 31.3 ms), and the two cancelled.

We note that this claim had already been falsified once by source inspection: SGLang's `token_usage` subtracts evictable blocks, so a parked session's prefix is genuinely not counted, and charging for it reserves capacity that really will be needed back. The end-to-end measurement agrees with the source reading.

E. Route 2: the growth reserve

Sessions grow by roughly 2.5K tokens per round, so the controller reserves capacity for the next `horizon` rounds. We swept the horizon over $\{0, 1, 2, 3\}$ and the reserve is not a free parameter to be minimised: Fig. 6 shows the response is non-monotone. Removing the reserve entirely costs 12.6 pp of SLO, and a reserve of two rounds is 15 minutes *slower* than three. The optimum is at one round, and there it touches the static front and stops—but that single point is the one this paper spends the most words on, because it does not reproduce: the repeat of this configuration lands 4.6 pp below the line (§V). Read the sweep as a shape, not as a result at its minimum.

F. Route 3: the hard ceiling

Lowering the budget fraction is the crudest available knob and behaves as the other routes do: it slides the operating point along the front. We report it only for completeness, since it adds no mechanism.

G. An independent adaptive controller

Every controller above is ours, which invites the objection that the negative result is a statement about one design rather than about adaptive admission. We therefore implemented a second, mechanistically different controller: an AIMD window over an agent-level admission set, following the control law of the congestion-based system we compare against in §VII. The window grows additively while cache occupancy sits below a low watermark, is cut multiplicatively when occupancy exceeds a high watermark *and* the prefix hit rate falls below a threshold, and holds otherwise. Cuts pause the youngest sessions at their next step boundary; growth resumes paused sessions first. Every input is a runtime gauge—occupancy and hit rate from the engine, nothing from the trace—and the window acts through the same named permits as Foyer.

It does not beat the static front either, at either parameter setting (Table I). At the values reported in that paper's evaluation it finishes in 364.3 minutes at 81.4% SLO, 11.2 pp below the envelope; the same arm with its retuned watermarks finishes in 264.6 minutes at 52.8%, 33.4 pp below. The retuned setting is also one of the most volatile runs we report: its segment-to-segment SD is 13.2 pp of SLO, with segment values swinging between 37 and 72. Three runs are worse (`cap4` at 15.8, the all-throttles-off arm at 14.4, and `breshz1` at 14.2), and no other controller arm is as bad—which is what a control loop oscillating around a capacity cliff looks like.

We report two things honestly about this comparison. First, this is our reimplementation of a published control law, not the authors' system, and a reimplementation of any controller can underperform its paper. We built it to the published law, gave it the same permit interface and the same 2 s cycle as Foyer, and did not tune it beyond the two documented settings; but the caveat stands. Second, both controllers stall. At its paper parameters this one pinned its window at one session, admitted nothing, and left the engine idle for over an hour. Our own controller does the same thing more briefly: in 4 of the 11 Foyer runs an audit found a single window of 150–175 s in which the engine was empty, the queue held 90 sessions, the SLO valve was open, and the controller issued no permit at all—about 0.9% of each run's wall clock. The remaining runs, including the best one, show no such window. Nothing is logged as an error and the run recovers on its own.

We traced one such window in a repeat of our best configuration and found a code path that explains it: single-flight admission considered only the head of the FIFO queue, and when the head was a session too large to fit the remaining budget it was never admitted, never left the queue, and blocked every smaller session behind it. That is the code every run in this paper executed. The fix in our repository—scan the queue, still admit at most one per cycle—is *not* in the artifact that produced these results, so the numbers here predate it and we do not claim them for it. We are also explicit that this is *a* path, not a proven cause: the audit is a log signature, and the same signature can be produced by other closed gates. What we can say is that the signature is present in a third of our Foyer runs and absent from all of our static caps, which have no admission decision to get stuck in.

That is the honest version of a finding we first reported too strongly. An intermediate draft of this paper called it a deadlock and said the configuration that produced our best result was the one that deadlocked on its repeat. Neither holds: the windows are 160 s and self-clearing, and the best configuration never showed one. The windows are also too small (0.9%) to move any verdict we report. We keep them because the mechanism is real and its size is the point: on a pool that holds four median sessions, a controller's failure mode is a silent stall, not graceful degradation, and it took a repeat run to see it.

H. Route 4: execution granularity

The first three routes all assume that what a controller can set is *how many sessions are resident*. A static cap sets the same quantity, so it is unsurprising that they land in the same place. The fourth route attacks the assumption instead: because a large fraction of live sessions are parked at any instant, a cap on *resident* sessions leaves the engine idle whenever all residents happen to be parked. Capping the *running-request* count instead should keep the engine busy while leaving the resident count high.

The permit design supports this directly: a paused session finishes its current round and blocks before its next one, and resumes when the controller re-admits it. We implemented it as a gate on the engine's own `num_running_reqs` and swept the gate over $\{2, 4\}$.

It does not work, and the measurement says why. The gate drove running concurrency from 1.42 to 1.29 and live sessions from 2.78 to 1.91: it tightened both rather than pinning the one it aims at. That is what we observe, and we state it as the observation it is rather than deriving a bound we cannot support. A live session spends most of its time parked in a tool wait (the ratio of mean running requests to mean live sessions is 0.38 to 0.66 across our arms), so pausing live

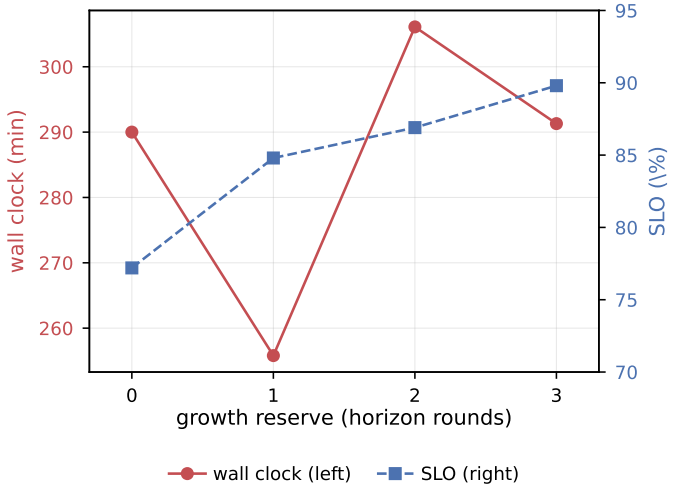


Fig. 6. The growth reserve is not a free parameter. Removing it costs 12.6 pp of SLO; two rounds is 15 minutes slower than three. The optimum at one round touches the static front and does not cross it—and is the one point here that does not reproduce on a repeat (it lands 4.6 pp below). The other three settings were each run once, so the shape is better supported than any single point on it.

sessions removes capacity that was not running in the first place—but that ratio is an average over a run, not a bound on any instant, and the running count can in principle reach the live count. Both repairs fail: without a constraint on admission, a paused session is re-admitted on the next cycle and blocks for at most one 2 s cycle; with one, the policy is a session cap again. The obstacle is granularity—`paused` takes effect at round boundaries, while pinning an instantaneous count requires control the permit mechanism does not have.

The measured outcome matches: the gate drove running concurrency from 1.42 to 1.29 and live sessions from 2.78 to 1.91. It tightened rather than pinned, and a tightening is another cap.

VI. Load Dependence

A negative result about adaptivity invites the obvious objection: adaptivity pays when conditions *change*, and a single stationary trace is the one case where a static cap should be near-optimal. We agree. What we test here is a weaker thing—two separate constant-load tiers—and §VIII says what that leaves open.

We run the same controller and the same cap family on a 25-session tier. The throttle configuration is reversed: with all safety throttles disabled, the 25-session tier runs *51% faster* (58.4 to 28.8 minutes) at a cost of only 2.4 pp of SLO, while on the 200-session tier disabling the same throttles costs 42% (291.3 to 415.0 minutes) and 43.7 pp of SLO. One switch, two regimes, opposite signs, both around 40–50% (Fig. 7). No single scalar setting is optimal across both.

We flag the generation gap rather than let the figure imply more than we ran. The two 200-session numbers are current; the two 25-session numbers are from an earlier harness revision, and we never ran the all-throttles-off arm on the 25-session tier under the current one. So the reversal is a direction we measured across two harness generations, not a same-generation paired comparison, and a reviewer should read it that way. The same-generation 25-session measurement we do have is in Table II, and it does not contain that arm.

TABLE II
THE 25-SESSION TIER, ALL FOUR ARMS UNDER THE CURRENT HARNESS. THIS IS THE SAME-GENERATION COMPARISON EARLIER DRAFTS OF THIS PAPER WERE MISSING; `cap4` AND THE FOYER ARM WERE RUN FOR THIS REVISION.

Policy	Wall (min)	Conc.	SLO (%)	Seg. SD (pp)
cap2	32.4	1.95	92.8	11.0
cap3	24.2	2.77	91.2	5.1
cap4	23.6	3.39	79.2	21.4
Foyer	27.6	2.47	92.8	7.1

This is the regime in which adaptive control should win, and we now have the same-generation measurement that earlier drafts of this paper were missing. Four arms on the 25-session tier, all under the current harness (Table II): static caps of two, three and four finish in 32.4, 24.2 and 23.6 minutes at 92.8%, 91.2% and 79.2% SLO, and Foyer finishes in 27.6 minutes at 92.8%.

The finding is a trade, not a defeat, and it is the opposite of what the 200-session tier shows. Against the same-generation envelope—which runs from `cap4` at (23.6, 79.2) through `cap3` at (24.2, 91.2) to `cap2` at (32.4, 92.8)—Foyer sits $+0.9$ pp above the line. A static cap of three is still 12% faster in wall clock, but it gives up 1.6 pp of SLO to do it, so neither dominates the other. On this tier the controller is a different point on the front rather than a dominated one, which is exactly what the 200-session tier denies it.

We are careful about the size, and about the yardstick. $+0.9$ pp is measured against a straight-line interpolation between static points, not against a switching policy anyone ran, so the defensible statement is that Foyer is not dominated by any static point we measured on this tier—not that it beats what a static mix could achieve. On 125 requests one request is worth 0.8 pp, so the margin is about one request. The tier's segment-to-segment SD (5.1 to 21.4 pp) is not the right yardstick: segment volatility is not run-to-run uncertainty. The right one is the replicate spread we have—which, across the four configurations we repeated, is 0.5, 1.0, 3.9 and 5.1 pp. Against the two tightest of those $+0.9$ pp is about a replicate wide; against the two widest—which belong to the configurations that decide our headline elsewhere—it is well inside one. We quote the tightest pair because it is the least favourable to us, not because it is the whole distribution. What we will defend is the weaker claim: on the low-load tier the controller is not worse, and the same switch that is worth $+51\%$ there costs -42% at high load. A controller that is competitive where load is light and dominated where load is heavy is the pattern this paper set out to explain, and it is the pattern we measured.

The second finding is the more useful one. The signal an

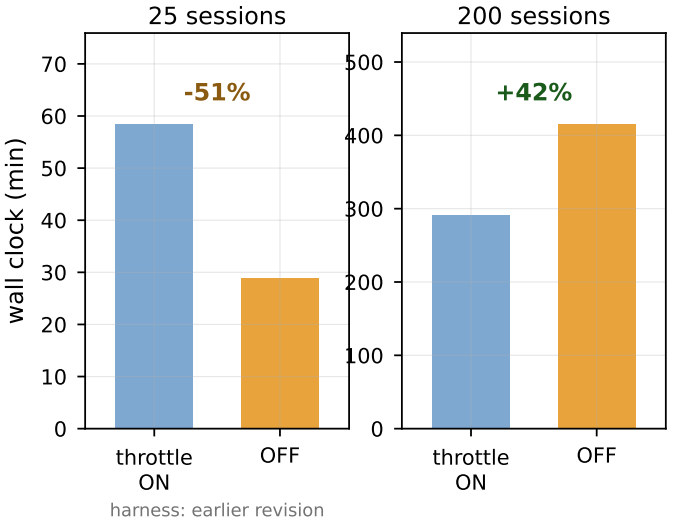


Fig. 7. The same throttle switch, two load tiers, opposite signs. The 25-session bars are from an earlier harness revision and are shown as a direction, not a magnitude—the current-harness 25-session comparison is in Table II instead. The 200-session bars are current.

adaptive controller would need is available—the number of sessions waiting for admission differs by 6–7× between the tiers (median 9–11 at 25 sessions against 66 at 200, maximum 135)—but its *interpretation* is not obvious. At the 25-session tier the controller holds eleven sessions back while running exactly one; that backlog is manufactured by the throttling itself rather than evidence that demand has slackened. Reading it correctly requires combining instantaneous backlog with progress through the run, which is a stateful computation a cap cannot perform. We regard this as the most promising direction we did not finish, and we are explicit that we did not finish it.

VII. Related Work

a) Admission and scheduling for agentic serving.: Recent systems report large gains from treating an agent session, rather than a request, as the schedulable unit, and from preserving session state across tool calls: session-aware admission with CPU-restorable KV [5], [6], program-level pause/resume with shortest-context-first restoration [7], and congestion-based control over an agent-level window [8]. A recurring measurement in this line is that recomputation, not admission, dominates: [8] attributes 49.1% of end-to-end latency to recomputation, and [9] preserves state on the CPU at the cost of a reload. Our results are consistent with this literature rather than in tension with it, and §VIII explains why: the mechanism these systems add is one our controller already has.

b) Engine-side techniques.: A separate line improves the engine rather than the admission decision. Iteration-level batching and paged KV [4], [10] are the substrate every policy here runs on; chunked prefill with stall-free batching [11], prefill/decode disaggregation [12], and cache-centric scheduling [13] are the refinements. Our §III measures 2.6–8.2 ms of each 27–37 ms step as work that is not decoding—prefill and non-decode overhead—which is exactly what this line attacks. We expect these techniques to move the frontier rather than to decide who sits above it, but we did not run the interaction and so state it as an expectation.

c) State retention and preemption.: A third line keeps session state alive across the gaps we measure in §III: keeping KV on cheaper storage across turns [14], [15], migrating it between instances [16], and preempting at iteration granularity [17]. These systems exercise finer control than our 2 s admission cycle, which is the boundary of our negative result: we falsify one control granularity, not the family. What they share with Foyer is that the state is not discarded—which is why, as we argue below, adding admission intelligence on top buys nothing here.

d) Application-level dependencies and replay.: Serving an LLM *application* rather than a request means the scheduler sees dependencies between calls [18], and agentic traces have structure that request-level benchmarks do not capture [19]. Our replay preserves that structure but fixes it in advance, which is exactly the limitation these works would press on.

e) Prefix caching.: The prefix cache is often proposed as the signal an admission policy should use. In our workload it is not a policy variable: round-1+ hit rate tracks the trace's own planned value to within a few points across the admission policies that keep the pool from overflowing—80.6% measured against 80.9% planned on the Foyer baseline, and 78.5% against the same 80.9% for `cap2`. The qualifier is load-bearing and we did not state it in an earlier draft: `cap5` admits five sessions to a pool that holds four, and its round-1+ hit rate is **32.8%**, not within a few points of 80.9%. So admission policy does move this quantity—it just does not move it for policies that stay under the pool's capacity. What is true is the narrower claim: among policies that do not oversubscribe, the prefix cache is not a lever they can pull. (Both numbers are the unweighted mean of the per-step planned hit rate over rounds 1 and later; the token-weighted figure for the same steps is 86.2%, and the all-steps figure is 73.8%. We use the unweighted round-1+ convention throughout, and flag it because the paper's earlier drafts mixed conventions.) We note the boundary of this claim—we varied admission policy, not eviction policy, and SGLang exposes alternate eviction policies we did not test. A characterization of production KV-cache traffic at a large cloud provider finds that eviction-policy design is "highly workload-dependent" and that reuse patterns differ sharply by request category [20]; that is reason to expect a single trace family to understate the space, though it is not a policy comparison and we do not present it as one.

VIII. Discussion

a) Why published gains do not reappear.: The systems cited above win by keeping session state alive across tool waits and by pausing at agent granularity rather than request granularity. Foyer has both properties by construction: the permit file gives per-session pause and resume, and the controller never frees capacity by discarding a session—it admits fewer

sessions instead. To that extent the mechanism these systems add is one our controller already has, which is one reason adding admission intelligence on top of it buys nothing here.

We are careful not to overstate this, because the distinction is between *choosing* to discard state and *having* it evicted. Retention here is the engine's radix cache doing its ordinary job; it is not something the controller implements, and §III measures that the cache evicts parked prefixes under pressure and that they are recomputed. What Foyer avoids is the deliberate trade of state for capacity. A system that manages the host tier as an explicit policy—as [9] and [14] do—holds a lever we do not, and our negative result does not cover it.

b) The threshold decides the verdict, and we report that.: Every SLO number in this paper counts a request as good if its time-to-first-token is under 10 s. That constant sets the whole vertical axis, so we re-derived the frontier at 5 s and 20 s as well, from the same per-request logs. The ordering of the static caps does not change. What changes is where the controllers sit relative to them, and at the loose end one of them crosses:

SLO threshold	5 s	10 s	20 s
`horizon`=1	-0.5	$+0.22$	$+2.7$
Foyer base	-2.1	-1.1	$+0.7$
`bres`	-11.9	-7.8	-4.3
Concur-style AIMD	-14.2	-11.2	-6.5

Two readings, and we give both. The first is that our headline claim is conditional: at the strict threshold that interactive serving actually cares about, no controller reaches the static front; at a loose one, one does. The second is that the crossing is still not a win, though the reason is more awkward than we first wrote. $+2.7$ pp at 20 s is larger than the replicate SLO spread of two of the three controller-side configurations we repeated (1.0 and 0.5 pp) but smaller than the best arm's own (3.9 pp) and smaller than an envelope anchor's (5.1 pp)—and it is the anchors, not the controllers, that set the line the margin is measured against. A third pair exists—two runs of `cap3`+HiCache—but one of them exceeded its own cap, so we do not use it as a replicate. A margin that appears only at one threshold and only at a size we cannot resolve is not evidence that adaptive admission works; it is evidence that the question needs more runs than we have. We state the conditional version in the abstract, and we flag this as the second thing we would re-run with more GPU time.

c) The SLO metric excludes the controller's own cost, and we checked.: Every SLO number above counts a request as good if its time-to-first-token is under 10 s, and the runner measures that from the moment the request is *submitted to the engine*. A session that waits for a permit has not been submitted yet, so the wait is invisible—and that wait is precisely the cost an admission controller imposes. We therefore recomputed the runs we have per-step logs for on a ready-to-first-token basis, where a round's clock starts when the session is actually able to run (its arrival for round 0, the previous round's completion plus its tool wait otherwise). That is nineteen of the twenty-three 200-session runs in Table I; the no-control baseline and the two runs whose per-step logs we did not recompute are absent, and we do not extend the claim to them.

The correction is large. Every configuration loses 3–19 pp of SLO, and the loss is systematic: tight caps queue the hardest, so `cap1` falls from 94.8% to 76.2% while `cap5` falls only from 27.1% to 24.2%. A session admitted last under `cap1` can wait hours for its turn; its engine-side TTFT is still 1.6 s.

What does *not* change is the verdict, within the subset we recomputed. The static envelope keeps the same four anchors, and of the eleven controller arms, nine move by no more than 2.7 pp and none crosses the ± 1.0 pp band into a new verdict. Two move more: the all-throttles-off arm improves by 8.3 pp and the retuned AIMD arm by 2.8 pp, both of which stay below the front either way. The one that decides our headline moves the wrong way for the controller—`horizon`=1 goes from $+0.22$ to -0.68 pp, still a tie but now on the below side.

The subset matters and we state its edge. The two configurations whose repeats decide how we read the verdict—`horizon`=1's repeat and the fastest anchor `cap4`+HC's—are not in the nineteen, because we did not recompute their per-step logs on the ready clock; the rebuilt envelope therefore rests on the original anchors without their repeats, and a ready-clock frontier that included them could move the line by the 3.9 and 5.1 pp those repeats differ by. We do not claim the verdict is unchanged under every SLO definition—only that on the runs we could recompute, moving the clock to the ready point costs every configuration 3–19 pp and changes no verdict in that set. We report this because a reviewer should ask it, and because the answer is the useful part: the negative result is not an artifact of choosing an SLO definition that flatters controllers. If anything the flattering choice was the one that let our best arm sit above the line.

d) Limitations.: Our results come from one 24 GB accelerator, one 7B model and one trace family; generality is untested. Only four configurations have been repeated—twice, twice, twice and three times—so the replicate spread we report in §V is estimated from one to two degrees of freedom per configuration; a margin as small as the one that decides our verdict would need several more repeats per configuration to resolve, and we did not run them. Two specific boundaries matter. First, the pool holds about four median sessions, so concurrency is capacity-bound by construction; on a device with a much larger pool the premise may not hold and the conclusion may invert—we regard reproducing on a larger device as the single most important next experiment. Second, the load is stationary, single-tenant and single-SLO. The load-dependence result of §VI is measured on two *separate* constant-load tiers, so it says what to expect when conditions differ, not what happens while they change; a trace whose load moves during the run is the experiment that would settle it, and we did not run one. Multi-tenant and heterogeneous-SLO settings are where we would look next.

e) Reproducibility.: The front and the signed distance of each controller are computed by `scripts/lab/frontier.py` from the raw per-step logs; the run tables and the four runs shorter than 999

steps are computed by `scripts/lab/gen_table.py` and `scripts/lab/run_summary.py` respectively, and §IV is computed by `gen_elasticity.py` and `gen_elasticity_hc.py`. Every run is guarded against a shrunken KV pool.

One caveat about the controller artifact, which a reader with our code would otherwise hit. The copy that executed every run in this paper is not byte-identical to the copy in our repository: the running one is `f825bc768188` and the repository's is `a21bd2b63774`, and the difference is exactly the queue-scan fix described in §V-H—the repository has it, the running copy does not. The numbers in this paper therefore describe the pre-fix path. We report the divergence rather than reconcile it, because re-running under the fixed controller is an experiment we have not done, and it is the one we would do first with more GPU time: if the queue scan removes the stall windows, the stalls are a bug we fixed; if it does not, they are a property of admitting one session at a time. We report six claims we retracted during the work, because the retractions were the most informative events. In order of how much they cost us:

The denominator of the step cost. Our first §III divided by the whole-run mean running count while multiplying by busy time. The two are not the same quantity—the whole-run mean includes the idle samples—so τ came out low by exactly the busy fraction, 36% for `cap1`, and two independent errors cancelled to make our two measurement methods appear to agree. This is the one that changed a headline number.

A mechanism refutation that was itself refuted: we withdrew the eviction-to-recomputation account on the strength of two checks that were both identities rather than measurements (§III).

A median-based reading of the tool wait that inverted a central assumption. *A monotone relation between idle fraction and concurrency* that the full data does not show. *A growth-sensitivity conclusion* that rested on an arm we had already invalidated. And *the accounting double-charge* of §V, falsified twice—once by source inspection and once by the end-to-end measurement here.

IX. Conclusion

We asked whether an adaptive admission controller can beat a tuned static concurrency cap on a memory-bound agentic LLM workload, and found that on our system it cannot: nine variants, four routes tested, none above the static front. We expected the reason to be a cancellation—admission policy controls how many requests are in flight, the per-step cost rises with that number, and the product is what the wall clock sees—and we built a two-part decomposition of the wall clock that makes the cancellation measurable. That decomposition holds, and the cost it measures is real, but it is not the whole reason and on the configuration our controllers run it is not the binding one: there the elasticity stays below one across the operating range, so added concurrency was still cheap and the engine had room. Our controllers did not take it. What we can say at the end is narrower than what we set out to say: on this hardware and this workload, the admission rules we could build and tune did not beat a static cap, the capacity estimate they act on is $1.79\times$ the truth, and a structural idle component that no policy we built recovers at a profit remains. The regime in which adaptivity should have paid—where the same switch is worth $+51\%$ and -42%—is one where the controller is not dominated by any static point we measured, and does not dominate one either. The measurement we offer is a diagnostic an engineer can take before committing to a sweep; whether it predicts the outcome on another workload is a question we tested once and could not confirm.

References

[1] K. Zhu, M. Jacob, C. Ma, Y. Pan, S. Wang, A. Krishnamurthy, and B. Kasikci, "TraceLab: Characterizing coding agent workloads for LLM serving," 2026, preprint; this paper's trace source.

[2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 62 557–62 583, arXiv:2312.07104; v1 title was "Efficiently Programming Large Language Models using SGLang".

[3] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, A. Yang, R. Men, F. Huang, S. Quan, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024.

[4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 611–626.

[5] U. Meng, U. Zhao, D. Li, G. Gao, and F. Du, "Talaria: Session-aware serverless serving of hundred-billion-parameter LLMs," 2026, preprint; author names as listed on arXiv.

[6] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," 2025, preprint, no venue.

[7] H. Kang, Z. Li, W. Xu, X. Yang, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, "ThunderAgent: A simple, fast and program-aware agentic inference system," 2026, preprint, no venue.

[8] Q. Chen, Z. Ye, T. Tang, P. Sun, B. Tian, G. Wang, S. Li, Y. Wen, Z. Han, and T. Zhang, "CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control," in *International Conference on Machine Learning (ICML 2026)*, 2026, poster, Seoul, July 2026; OpenReview id U7c6glPdDW; arXiv:2601.22705.

[9] T. Xia, H. Li, Z. Li, X. Chen, H. Kang, Y. Qiao, Y. Xu, and I. Stoica, "Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI," 2026, preprint, no venue.

[10] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, 2022, pp. 521–538.

[11] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 117–134.

[12] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 193–210.

[13] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *ACM Transactions on Storage*, vol. 22, no. 4, pp. 1–38, 2026.

[14] L. Yu, J. Lin, and J. Li, "Stateful large language model serving with Pensieve," in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025, pp. 144–158.

[15] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in *2024 USENIX Annual Technical Conference (USENIX ATC '24)*. USENIX Association, 2024, pp. 111–126.

[16] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 173–191.

[17] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, "FastServe: Iteration-level preemptive scheduling for large language model inference," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026, pp. 57–74.

[18] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 929–945.

[19] M. Wang, Y. Yue, S. Li, Y. E. Zhou, C. Wang, and J. Huang, "Benchmarking LLM serving systems for agentic AI workloads with XPerf," 2026, preprint.

[20] J. Wang, J. Han, X. Wei, S. Shen, D. Zhang, C. Fang, R. Chen, W. Yu, and H. Chen, "KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider," in *2025 USENIX Annual Technical Conference (USENIX ATC '25)*. USENIX Association, 2025, pp. 465–482.